

SQL

O que é e como usar os principais comandos básicos

O SQL é uma linguagem padrão para manipulação de registros em [bancos de dados relacionais](#). A sigla SQL vem dos termos em inglês “Structured Query Language”, que podem ser traduzidos para o português como “Linguagem de Consulta Estruturada”.

Trata-se ainda de uma linguagem declarativa que apresenta sintaxe simples e fácil de ser aprendida até por pessoas iniciantes na área da programação.

Como surgiu o SQL? História e criador!

A linguagem SQL surgiu a partir de um estudo realizado na década de 70 pelo laboratório da IBM. Esse estudo tinha como um de seus objetivos elaborar uma linguagem que fosse capaz de simplificar o gerenciamento de informações em bancos de dados do tipo relacional (baseado em tabelas).

Foi aí que surgiu a especificação original do SQL, que foi feita por dois conhecidos cientistas da [computação](#), **Donald D. Chamberlim** e **Raymond F. Boyce**. O sucesso da linguagem foi imediato e rapidamente diversos “dialetos” começaram a ser desenvolvidos, o que gerou a necessidade do ANSI (American National Standards Institute) padronizar o seu uso.

Assim, o SQL continua sendo usado em grande parte dos bancos de dados relacionais até os dias atuais. Mas, apesar da padronização da linguagem, ainda existem algumas variações utilizadas por diferentes SGBDs (Sistemas Gerenciadores do Bancos de Dados).

Para que serve e onde o SQL é usado?

O SQL é usado para fazer qualquer tipo de manipulação dos registros de um banco de dados. Ou seja, ele serve para **criar, inserir, atualizar, excluir e consultar as informações armazenadas na base**, além de outras diversas funções mais complexas.

Atualmente, vários SGBDs amplamente utilizados no mercado são baseados em SQL. Isso acontece porque bancos do tipo relacional armazenam as informações em estruturas semelhantes a tabelas, permitindo que sejam criadas relações entre elas. A linguagem SQL foi criada justamente para facilitar o gerenciamento de dados armazenados em bancos que seguem esse padrão.

Alguns dos principais sistemas que utilizam essa tecnologia são:

- **MySQL**: criado pela Oracle, é uma opção que oferece serviços gratuitos e pagos;
- **PostgreSQL**: muito usado em [aplicações web](#), essa é uma opção gratuita de [código aberto](#);
- **Oracle**: conhecido pela sua [segurança](#), esse sistema é um dos mais usados por grandes corporações;
- **SQL Server**: desenvolvido pela Microsoft, ele oferece tanto serviços pagos quanto versões gratuitas para download.

Quais as vantagens e desvantagens da linguagem SQL?

E aí, conseguimos deixar evidente para que serve o SQL? Então, agora, confira quais são as vantagens e desvantagens que podemos encontrar nessa linguagem:

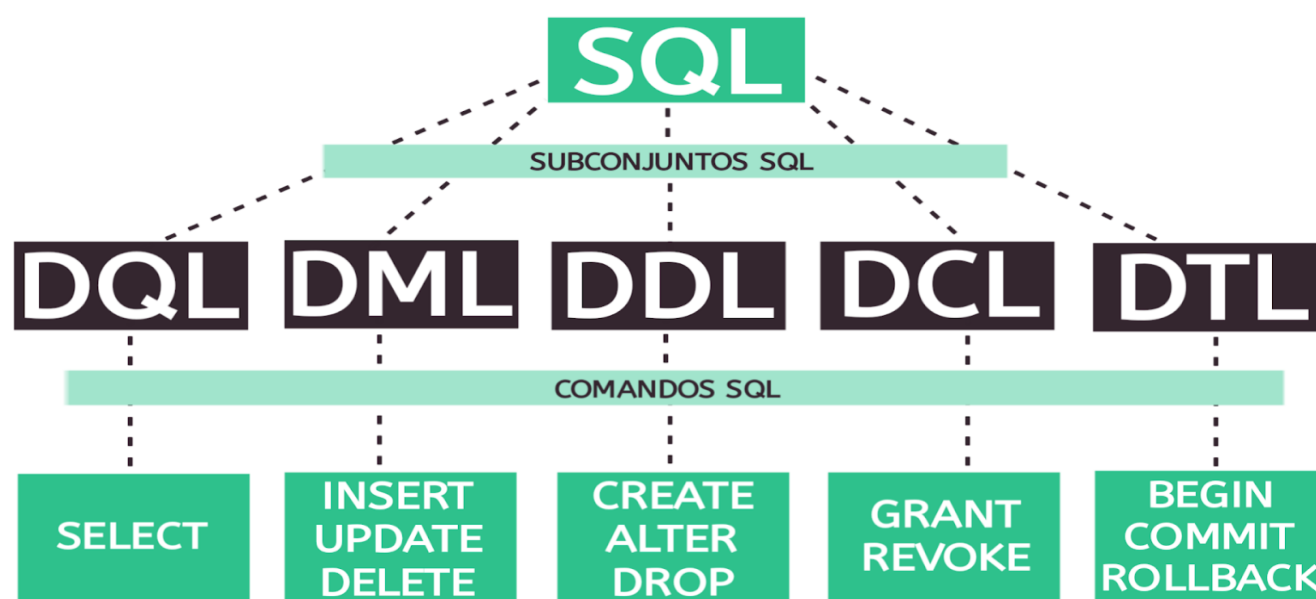
Principais vantagens

- **Padronização:** como dissemos mais acima, o SQL foi padronizado pelo ANSI. Por isso, ela oferece uma ampla documentação e, aprendendo a linguagem, você vai conseguir lidar facilmente com diferentes bancos de dados que a implementam;
- **Rápido acesso aos dados:** SQL permite a recuperação de múltiplos registros com um único comando, tudo feito de forma rápida e eficiente;
- **Portabilidade:** o SQL pode ser usado em diversas aplicações para diferentes plataformas;
- **Múltiplas visões de dados:** com SQL é possível definir diferentes visualizações da estrutura do banco para diferentes pessoas usuárias;
- **Linguagem cliente/servidor:** com SQL é possível implementar arquitetura cliente-servidor.

Principais desvantagens

- **Dificuldade com a interface:** para quem não está acostumado, a interface de um banco de dados SQL pode ser mais difícil de lidar;
- **Custo:** o custo operacional de alguns serviços oferecidos por bancos que utilizam SQL é caro e dificulta o acesso dos desenvolvedores.

Conheça os 5 subconjuntos da linguagem SQL



Tipos de Dados

Os tipos apresentados abaixo fazem parte do conjunto de tipos do padrão ANSI 92 da linguagem SQL.

CHAR(n)	Armazena caracteres alfanuméricos de tamanho fixo n.
VARCHAR(n)	Cadeia de caracteres de comprimento variável e tamanho máximo de n caracteres.
INTEGER	Dado numérico inteiro de tamanho fixo.
DECIMAL (n, m)	Dado numérico de tamanho variável, sendo n o número total de dígitos e m o número de casas decimais.
NUMERIC (n,m)	
BIT(n)	Sequência n de bits.

TIME	Hora de tamanho fixo.
DATE	Data de tamanho fixo.

Subconjuntos da linguagem SQL e seus comandos

A linguagem SQL apresenta uma organização estrutural muito bem definida que divide seus comandos em 5 subconjuntos diferentes. Cada um desses conjuntos apresenta objetivos específicos que explicaremos a seguir. Observe!

DML: linguagem de manipulação de dados

O DML (Data Manipulation Language) é o subconjunto do SQL que **define os comandos usados para manipular os dados armazenados em um banco**. Esse é um dos conjuntos mais utilizados, pois ele fornece operadores que nos permitem inserir, excluir e alterar os registros de uma tabela, por exemplo. Os comandos mais importantes desse subconjunto são: INSERT, DELETE e UPDATE.

DQL: linguagem de consulta de dados

O DQL (Data Query Language) é o conjunto SQL que define o comando mais popular da linguagem, o SELECT. Esse comando é essencial para que possamos **consultar os dados que armazenamos em nosso banco**.

DDL: linguagem de definição de dados

O DDL (Data Definition Language) é o subconjunto SQL que apresenta **comandos usados para gerenciar as estruturas do banco de dados**. Com ele, podemos criar, atualizar e remover objetos da base, como tabelas e índices. Os comandos definidos pelo DDL são: CREATE, DROP e ALTER.

DCL: linguagem de controle de dados

O DCL (Data Control Language) é o subconjunto no qual encontramos **comandos para controlar o acesso aos dados da nossa base**. Utilizando esse conjunto, conseguimos estabelecer restrições e permissões para quem acessa o banco por meio dos comandos GRANT e REVOKE.

DTL ou TCL: linguagem de transação de dados

O DTL (Data Transaction Language) — também conhecido como TCL (Transaction Control Language) — é o subconjunto SQL que define comandos que utilizamos quando é necessário **gerenciar transações feitas no banco**. Isso significa que eles permitem iniciar, confirmar e desfazer determinadas alterações. Os comandos estabelecidos pelo conjunto são COMMIT, BEGIN e ROLLBACK.

Principais comandos, funções e operadores do SQL!

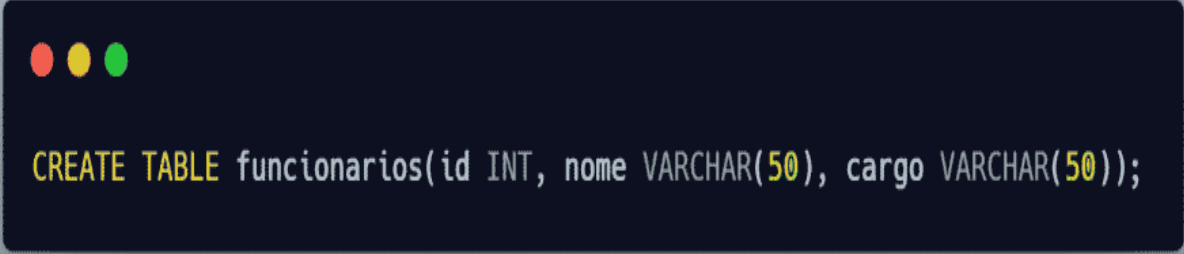
Agora que você já entendeu como os [comandos SQL](#) são divididos entre os 5 subconjuntos da linguagem, vamos nos aprofundar mais um pouquinho no assunto e mostrar para que serve cada um deles. Confira!

SQL CREATE

O comando CREATE, conforme seu nome indica, serve para **criar o banco de dados e as tabelas que ele contém**. A instrução usada para isso é bem simples:

CREATE DATABASE empresa;

Com o nosso banco criado, podemos começar a criar as tabelas em que vamos armazenar os registros. Para isso, usamos o comando CREATE TABLE, observe:



```
CREATE TABLE funcionarios(id INT, nome VARCHAR(50), cargo VARCHAR(50));
```

É importante observar que, ao criar a tabela, também estamos especificando as colunas que ela vai conter, os tipos de dados que cada uma vai suportar e o comprimento máximo para os campos do tipo VARCHAR.

SQL INSERT

O comando INSERT é usado para **inserir novos registros em uma tabela do banco**. Agora que nossa tabela “funcionarios” já foi criada, podemos usar uma estrutura básica dessa instrução para começar a popular o banco. Observe:

```
INSERT INTO funcionarios (id, nome, cargo) VALUES (1, 'Fernando', 'Analista de Sistemas');
```

Nesse breve exemplo que mostramos, você deve reparar que “funcionarios” é o nome da tabela e “id”, “nome” e “cargo” são as colunas da tabela que estamos adicionando novos dados.

SQL UPDATE

Já o comando UPDATE é utilizado para **atualizar os registros que foram armazenados no banco**. Essa declaração é usada junto com a cláusula WHERE, que especifica exatamente a linha da tabela que terá seus dados alterados.

No exemplo acima, adicionamos um novo registro na tabela “funcionarios”, mas agora queremos atualizar um dos dados que inserimos, mais precisamente o cargo que foi especificado. Para isso, usamos o comando:

```
UPDATE funcionarios SET cargo = 'Analista de Sistemas e Aplicações' WHERE id = 1;
```

SQL DELETE

O comando DELETE, como você já deve imaginar, é usado para **excluir dados de uma tabela**. Assim como o anterior, ele também é usado junto com a cláusula WHERE. Nesse caso, vamos imaginar que não precisamos mais das informações que inserimos nos exemplos acima e queremos apagá-las da nossa tabela. Podemos fazer isso da seguinte forma:

```
DELETE FROM funcionarios WHERE id = 1;
```

SQL SELECT

O comando SELECT permite definir critérios para **realizar consultas aos registros que foram armazenados no banco de dados**. Nesse caso, a forma mais simples de busca que podemos fazer é consultar todos os campos de uma tabela, dessa forma:

```
SELECT id, nome, cargo FROM funcionarios;
```

Entretanto, existem diversas outras maneiras de realizar consultas e, utilizando os operadores corretos, podemos deixar nossas buscas mais refinadas.

Para ficar mais evidente, imagine que você quer consultar o nome do funcionário com id = 15. Novamente, para especificar exatamente qual é o dado que queremos visualizar, utilizamos a cláusula WHERE, conforme mostramos no comando abaixo:

```
SELECT nome FROM funcionarios WHERE id = 15;
```

Unindo tabelas baseado na relação entre colunas: SQL JOIN

Como dissemos, muitas vezes é necessário fazer buscas mais complexas na base de dados e podemos utilizar diversas instruções para refinar nosso SELECT. Entre elas, estão as cláusulas JOIN.

Com elas, conseguimos **combinar linhas de várias tabelas com base na relação existente entre as colunas dessas tabelas**. Para ficar mais nítido, observe para qual caso cada uma delas é usada:

- **INNER JOIN**: retorna os registros que têm valores correspondentes em ambas as tabelas;
- **LEFT JOIN**: retorna todos os registros da tabela esquerda, além dos registros com valores correspondentes na tabela direita;
- **RIGHT JOIN**: retorna todos os registros da tabela direita, além dos registros com valores correspondentes na tabela esquerda;
- **FULL JOIN**: retorna todos os registros de ambas as tabelas quando há uma correspondência entre elas.

Unindo o resultado de duas ou mais tabelas: SQL UNION

A função UNION é utilizada para **combinar o conjunto de resultados de dois ou mais comandos SELECT**. No entanto, precisamos seguir algumas regras para que a declaração funcione. Elas são:

- cada SELECT deve ter o mesmo número de colunas;
- as colunas de cada SELECT devem estar na mesma ordem;
- os tipos de dados das colunas devem ser iguais.

Seguindo esses critérios, você pode usar o operador dessa forma:

```
SELECT nome FROM funcionarios
```

```
UNION
```

```
SELECT nome FROM clientes;
```

Avaliando uma lista de condições e retornando um resultado: SQL CASE

A expressão CASE é usada para **definir condições que devem ser atendidas durante a execução de um comando**. Várias condições podem ser estabelecidas com essa expressão, mas assim que uma delas for atendida, o comando vai retornar o resultado e as demais condições não serão lidas.

Para exemplificar, imagine que uma empresa vai estabelecer um bônus para as pessoas colaboradoras de acordo com o desempenho em um treinamento interno. Nesse caso, quanto maior a nota no treinamento, maior será a porcentagem obtida de bônus.

Em uma situação desse tipo, poderíamos usar a expressão CASE dessa forma:

```
SELECT nome, nota
CASE
WHEN nota = 7 THEN 100*0.7
WHEN nota = 8 THEN 100*0.8
WHEN nota = 9 THEN 100*0.9
WHEN nota = 10 THEN 100
ELSE 0
END AS bonus
FROM treinamento;
```

Pesquisando valores similares em uma coluna: SQL LIKE

O operador LIKE é utilizado para **encontrar um padrão em diferentes strings**. No entanto, para usá-lo, precisamos do auxílio de mais dois elementos, são eles:

- sinal de porcentagem (%): serve para indicar a posição em que o padrão procurado deve estar na string;
- sinal de sublinhado (_): serve para representar uma quantidade de caracteres.

Assim, esses caracteres podem indicar padrões diferentes de acordo com a forma que são aplicados em uma expressão. Alguns exemplos são:

- **LIKE 'a%'**: retorna qualquer string que inicie com a letra "a";
- **LIKE '%Gomes'**: retorna qualquer string que termine com "Gomes";
- **LIKE '_b%'**: retorna qualquer string que tenha a letra "b" na segunda posição;
- **LIKE 'd__%'**: retorna qualquer string que inicie com a letra "d" e tenha pelo menos mais 3 caracteres depois.

Para facilitar o entendimento, observe um exemplo de aplicação do operador LIKE em um comando SELECT:

```
SELECT * FROM funcionarios WHERE nome LIKE 'a%';
```

Observe que a instrução acima vai retornar os registros de todos os funcionários que têm o nome iniciado com a letra "a".

Selecionando valores entre um intervalo determinado: SQL BETWEEN

O operador BETWEEN serve para **selecionar os valores dentro de um determinado intervalo**, sendo que esses valores podem ser strings, datas ou números.

Pensando em uma aplicação prática, imagine que uma empresa precisa consultar os registros dos funcionários com salário entre 2000 e 5000 reais. Nesse caso, poderia ser utilizada a seguinte consulta:

```
SELECT * FROM funcionarios WHERE salario BETWEEN 2000 AND 5000;
```

Por outro lado, se fosse necessário consultar os registros fora desse intervalo, poderíamos usar o comando:

```
SELECT * FROM funcionarios WHERE salario NOT BETWEEN 2000 AND 5000;
```


Agrupando linhas com os mesmos valores: SQL GROUP BY

A cláusula GROUP BY é responsável por **agrupar linhas que apresentam valores correspondentes em determinadas colunas**. Muitas vezes utilizamos essa cláusula junto com funções de agregação do SQL, como MIN, MAX, COUNT e SUM, para encontrar somas, médias ou a frequência de vezes que um determinado valor aparece.

Por exemplo, vamos imaginar que para fazer um balanço de dados dos setores de uma empresa, precisamos identificar quantos funcionários atuam em cada cargo. Para esse propósito, o comando que poderia ser usado é:

```
SELECT COUNT(id), cargo FROM funcionarios GROUP BY cargo;
```

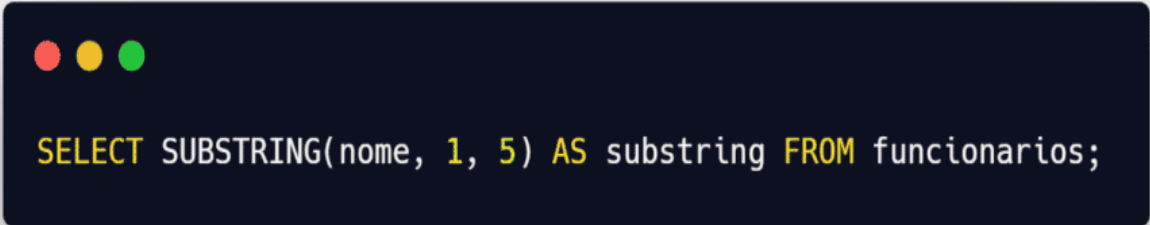
Após rodar essa instrução, será listada uma tabela com os cargos da empresa e a quantidade de funcionários que cada um apresenta.

Extraindo a parte específica de uma string: SQL SUBSTRING

A função SUBSTRING é usada para **retornar alguns caracteres de uma determinada string**. Para isso, precisamos indicar os seguintes argumentos na função:

- a string que desejamos extrair uma parte;
- a posição em que começaremos a extrair a substring;
- o comprimento da substring que queremos.

Para exemplificar, observe o comando abaixo:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It displays the SQL command: `SELECT SUBSTRING(nome, 1, 5) AS substring FROM funcionarios;`

```
SELECT SUBSTRING(nome, 1, 5) AS substring FROM funcionarios;
```

Perceba que, com a instrução acima, estamos usando a tabela “funcionarios” para retornar 5 caracteres da coluna “nome” começando pela posição 1. Além disso, as substrings geradas serão listadas em uma coluna chamada “substring”.

Contando valores de uma tabela de acordo com critérios específicos: SQL COUNT

Utilizando a função COUNT é possível **contar a quantidade de registros que correspondem a um critério específico**. Podemos, por exemplo, descobrir quantos funcionários temos em nossa tabela. O comando que usamos nesse caso é:

```
SELECT COUNT(id) FROM funcionarios;
```

Observe que no exemplo estamos especificando a coluna “id” para fazer a contagem. Também é possível utilizar * ao invés de informar uma coluna na função COUNT, mas é preciso cuidado ao utilizar essa forma, pois ela contabiliza todas as linhas, incluindo as com valores do tipo nulo.

Listando apenas os valores diferentes em uma coluna: SQL DISTINCT

A cláusula DISTINCT é responsável por **retornar apenas valores únicos em uma coluna**. Por isso, ela é bastante útil quando queremos eliminar resultados duplicados em uma consulta.

Pensando na nossa tabela “funcionarios”, vamos imaginar que nela há uma coluna para armazenar as cidades em que eles vivem e queremos saber quais são. Porém, muitos deles moram nas mesmas cidades e queremos simplificar nossa consulta.

Para isso, podemos usar a cláusula DISTINCT para eliminar as cidades duplicadas, desta forma:

```
SELECT DISTINCT cidade FROM funcionarios;
```

SQL DROP

Com o comando DROP é possível **excluir permanentemente os componentes de um banco de dados ou o próprio banco completo**. Existem outros comandos SQL que são usados para apagar dados da base, mas a diferença do DROP é que ele exclui tudo de uma vez.

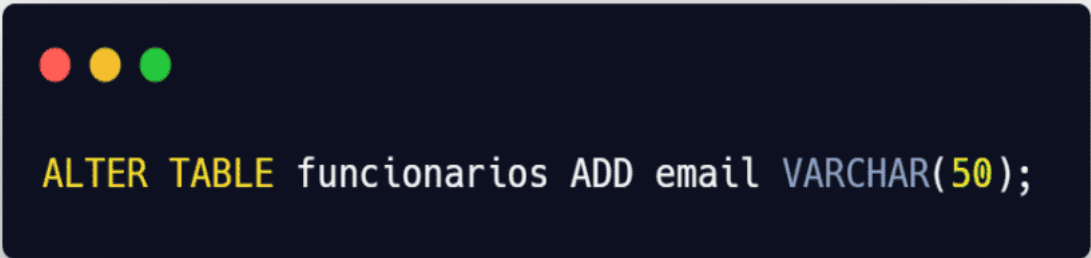
Se utilizarmos ele para remover uma tabela, por exemplo, todo o esquema da tabela será apagado, incluindo colunas, índices, permissões, restrições e, claro, os registros armazenados nela. A seguir, veja um exemplo de uso desse comando no qual deletamos a tabela “funcionarios”:

```
DROP TABLE funcionarios;
```

SQL ALTER

O comando ALTER é responsável por **modificar a estrutura de objetos armazenados no banco**. Ou seja, com ele podemos, por exemplo, adicionar ou remover colunas de uma tabela e ainda alterar o tipo de dados especificado para uma coluna.

Voltando ao exemplo da tabela “funcionarios”, imagine que precisamos **incluir uma nova coluna** na tabela para armazenar o e-mail de contato. Nesse caso, usaríamos o comando:



```
ALTER TABLE funcionarios ADD email VARCHAR(50);
```

Voltando ao exemplo da tabela “funcionarios”, imagine que precisamos **excluir uma coluna** da tabela. Nesse caso, usaríamos o comando:

Sintaxe: ALTER TABLE nome_tabela DROP COLUMN nome_coluna;

```
SQL > ALTER TABLE funcionarios DROP COLUMN email;
```

Excluir colunas usando Transact-SQL

Você pode excluir colunas usando Transact-SQL no SSMS, [Azure Data Studio](#) ou ferramentas de linha de comando, como o [utilitário sqlcmd](#).

O exemplo a seguir mostra como excluir a coluna.

```
ALTER TABLE dbo.doc_exb DROP COLUMN column_b;
GO
```

Se a coluna contiver restrições ou outras dependências, uma mensagem de erro será retornada. Resolva o erro excluindo as restrições referenciadas.

SQL TRUNCATE

O comando TRUNCATE é usado para **remover rapidamente todas as linhas de uma tabela**, mas sem deletar a estrutura da tabela em si. Uma vez que esse comando é rodado, a operação não pode ser revertida, portanto é preciso cuidado ao utilizá-lo.

A seguir, mostramos a instrução que seria usada para apagar todas as informações armazenadas na tabela “funcionarios”:

```
TRUNCATE TABLE funcionarios;
```

SQL COMMENT

Em SQL também é possível **usar comentários para explicar queries ou impedir que trechos de comando sejam executados**. Assim como outras linguagens de [programação](#), podemos fazer comentários de uma linha ou com várias linhas.

Comentários de linha única são iniciados com o símbolo —, conforme mostramos a seguir:

```
-- Escreva seu comentário aqui
```

Já comentários de múltiplas linhas utilizam o símbolo /* no início e são finalizados com */. Durante a execução do comando, qualquer coisa escrita entre esses caracteres será ignorada. Veja um exemplo:

```
/*Aqui temos
um comentário de
múltiplas linhas*/
```

SQL RENAME

Com o comando RENAME é possível **renomear tabelas do banco**. Entretanto, nem todos SGBDs suportam esse comando, já que ele não é padronizado. Apesar disso, sua sintaxe é bastante simples e podemos usá-lo dessa forma:

```
RENAME TABLE funcionarios TO funcionariosTI;
```

SQL GRANT

Com o comando GRANT é possível **conceder permissões específicas para determinados objetos do banco**. Ou seja, podemos especificar, por exemplo, que apenas um determinado usuário do sistema possa atualizar e inserir novos dados em uma tabela.

Para esse propósito, na nossa tabela “funcionarios” usaríamos a seguinte instrução para conceder privilégios a uma pessoa cadastrada com o nome “Roberta” :

```
GRANT INSERT, UPDATE ON funcionarios TO Roberta;
```

SQL REVOKE

Já utilizando o comando REVOKE você pode **remover os privilégios que foram concedidos com o GRANT**. Nesse caso, para remover da usuária “Roberta” a permissão para atualizar registros na tabela “funcionarios”, poderíamos usar o comando:

```
REVOKE UPDATE ON funcionarios FROM Roberta;
```

SQL BEGIN/SET TRANSACTION

O comando `BEGIN TRANSACTION` **indica o início de uma transação**, portanto os comandos que vierem abaixo dele vão pertencer à mesma transação. Para especificar o início de uma transação no seu código, basta incluir a seguinte declaração antes dos comandos que devem ser executados:

BEGIN TRANSACTION;

`SET TRANSACTION` também inicia uma transação, a diferença é que ele define características para a transação em questão. Ele pode, por exemplo, especificar se a transação será somente de leitura ou de leitura e gravação.

Para iniciar uma transação somente de leitura na nossa tabela “funcionarios”, poderíamos usar o comando abaixo:

```
SET TRANSACTION READ ONLY; -- aqui especificamos que a transação será apenas de leitura
```

```
SELECT nome, cargo FROM funcionarios WHERE id = 5
```

```
COMMIT;
```

SQL COMMIT

O comando `COMMIT` é usado para **indicar o fim de uma transação**, por isso ele sempre acompanha o comando `BEGIN`. Quando esse comando é rodado, todas as queries entre ele e o `BEGIN` serão salvas na transação atual. Já o que vier depois do `COMMIT` não fará parte dessa transação.

Observe o exemplo abaixo para entender melhor como utilizá-lo:

```
BEGIN TRANSACTION --aqui iniciamos a transação
```

```
UPDATE funcionarios SET cargo = 'Engenheiro de Software' WHERE id = 13;
```

```
COMMIT; --aqui encerramos e salvamos as alterações da transação
```

SQL ROLLBACK

Já o comando `ROLLBACK` também é usado para finalizar uma transação, mas nesse caso ele **indica que todas as alterações feitas na transação devem ser descartadas**, restaurando a transação ao seu estado original. Por isso, ele é muito útil quando alteramos algo que não deveria ter sido modificado.

Então, caso precise retornar alguma transação para o seu estado anterior, basta rodar esse comando em seguida:

```
ROLLBACK;
```

SQL SAVEPOINT

O comando `SAVEPOINT` também serve para manipular transações, mas nesse caso ele estabelece um “ponto de segurança” dentro da transação. Isso significa que **todos os comandos executados após um SAVEPOINT podem ser descartados com um ROLLBACK** sem que toda a transação seja descartada.

Para isso, precisamos criar o ponto de salvamento dentro da transação da seguinte forma:

```
SAVEPOINT nome_do_ponto_de_salvamento;
```

Já para descartar as alterações feitas depois de um `SAVEPOINT` basta rodar:

```
ROLLBACK TO SAVEPOINT nome_do_ponto_de_salvamento;
```

Conforme foi mostrado ao longo deste conteúdo, a linguagem SQL é de extrema importância para quem trabalha com tecnologia. Além de ser essencial para ampliar suas oportunidades no mercado, ela também é simples de aprender e de manipular, oferecendo acesso rápido e seguro aos dados de um sistema.

```
/* INSERIR TABELA
```

```
create table Tbl_Paciente (
CPF char(12),
Nome varchar(30),
Endereco varchar(50),
Sexo varchar(1),
Dt_Nascimento date);*/
```

```
--INSERÇÃO DE DADOS NA TABELA
```

```
--INSERT INTO Tbl_Paciente (cpf, nome, endereco, sexo, Dt_Nascimento)
--VALUES ('123456789100', 'Fernando da Silva', 'Rua 10, nr. 100 - Industrial', 'M', '2023-04-20');
```

```
--VISUALIZAR TODOS OS DADOS
```

```
--Select * from Tbl_paciente;
```

```
--select nome, cpf, sexo from tbl_paciente;
```

```
--Atualizar dados
```

```
--UPDATE tbl_paciente SET cpf = '999999999999';
```

```
--select nome, cpf, sexo from tbl_paciente;
```

```
--EXCLUSÃO DE DADOS DELETE
```

```
--DELETE FROM tbl_paciente WHERE Sexo='M';
```

```
--ALTERAÇÃO DA TABELA (Inserir/Excluir colunas)
```

```
--ALTER TABLE TBL_PACIENTE ADD CONTATO VARCHAR(20);
```

```
--ALTER TABLE TBL_PACIENTE drop column CONTATO;
```

```
--EXCLUSÃO DE COLUNAS PELO Transact-SQL
```

```
--ALTER TABLE dbo.doc_exb DROP COLUMN column_b;
```

```
--GO
```

```
/*create table funcionario
```

```
(matricula decimal(5) NOT NULL PRIMARY KEY,
```

```
nome char(30) NOT NULL,
```

```
rg decimal(9) NOT NULL UNIQUE,
```

```
sexo char(1),
```

```
depto decimal(5),
```

```
endereco varchar(40),
```

```
cidade varchar(20) DEFAULT 'São Paulo',
```

```
salario decimal(10,2) );*/
```